МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»
Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

по дисциплине «Жизненный цикл разработки программного обеспечения»

Тема: «Изучение распределенной системы контроля версий Git.
Ветвление, работа с удаленным репозиторием и слияния»

Выполнил:
студент группы 434701
Шурхай Н.А.

Проверил:
преподаватель кафедры ЭВМ
Внук О.В.

Минск 2026

1. ЦЕЛЬ РАБОТЫ

Целью настоящей лабораторной работы является практическое освоение архитектуры и базового инструментария распределенной системы контроля версий Git, изучение принципов командной и индивидуальной разработки программного обеспечения, а также приобретение устойчивых навыков управления версиями исходного кода в среде GitHub.

В процессе выполнения работы решаются следующие ключевые инженерные задачи:

- Криптографическая авторизация: настройка безопасного протокола SSH для аутентификации на удаленном хостинге репозиториев GitHub с использованием современного алгоритма эллиптических кривых Ed25519.
- Управление конфигурацией Git: глобальное и локальное конфигурирование профиля разработчика, интеграция системных утилит и устранение платформозависимых коллизий путей в среде Windows.
- Модели репликации и ветвления: освоение концепции серверного форка (Fork) и клонирования (Clone) репозитория организации, практическое применение модели Forking Workflow.
- Управление состоянием рабочего дерева: изучение переходов состояний файлов между Working Tree (рабочая директория), Staging Area / Index (индекс подготовки) и Git Repository (хранилище коммитов).
- Фильтрация артефактов сборки: проектирование и тонкая настройка конфигурационного файла .gitignore для подавления попадания временных файлов сред разработки (Microsoft Visual Studio, MSVC) и бинарных исполняемых файлов в историю ревизий.
- Интеграция кодовой базы: фиксация материалов предыдущей лабораторной работы в изолированной ветке lab1 и подготовка чистого шаблона проекта в ветке ProjectTemplate.
- Слияние и разрешение коллизий: освоение механизмов объединения параллельных ветвей разработки (Fast-Forward и Three-Way Merge), верификация сетевого графа коммитов (Network Graph).

2. ТРЕБОВАНИЯ И ИСХОДНЫЕ ДАННЫЕ

В рамках дисциплины «Жизненный цикл разработки программного обеспечения» для выполнения лабораторного практикума были установлены следующие исходные условия и нормативные требования:

- Организационный репозиторий кафедры: исходный проект BSUIR-SDLC/SDLC, содержащий нормативные отраслевые шаблоны технической документации программных систем (SRS по IEEE 830, SDD по IEEE 1016, архитектурные решения ADR).
- Персональный форк студента: репозиторий akutkola-source/SDLC, развернутый в личном профиле GitHub студента группы 434701 Шурхая Н. А.
- Используемый программный стек: система контроля версий Git версии 2.x, терминал Windows Command Prompt / PowerShell, встроенный криптографический сервис Windows OpenSSH, компилятор MSVC C++, среда Microsoft Visual Studio.
- Структура изолированных ветвей разработки: в соответствии с заданием в репозитории формируются две независимые ветки: lab1 (для изоляции исходных текстов виджета первой лабораторной работы) и ProjectTemplate (для размещения шаблона проекта и оформления файла описания README.md).
- Итоговый результат: защита выполненной конфигурации перед преподавателем, успешное слияние ветки ProjectTemplate в основную ветку main и публикация итоговой непротиворечивой истории коммитов.

3. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ GIT

3.1. Архитектурная модель данных и внутренние объекты Git

В отличие от классических централизованных систем управления версиями (таких как SVN или CVS), которые хранят информацию в виде дельт (наборов различий между последовательными версиями файлов), Git оперирует концепцией снимков (snapshots) целостного состояния файлового дерева проекта. Модель данных Git представляет собой направленный ациклический граф (Directed Acyclic Graph, DAG), вершинами которого выступают иммутабельные объекты, адресуемые по содержимому (Content-Addressable Storage).

Хранилище Git идентифицирует каждый объект уникальным 160-битным или 256-битным криптографическим хэшем (SHA-1 / SHA-256). В структуре каталога .git выделяются четыре базовых типа объектов:

- BLOB (Binary Large Object): объект, содержащий исключительно «сырые» двоичные или текстовые данные содержимого файла. Имя файла, права доступа и метаданные в BLOB не сохраняются.
- Tree (Дерево): объект, выполняющий роль каталога файловой системы. Он содержит список ссылок на другие объекты Tree (поддиректории) и BLOB (файлы), ассоциируя их с именами и атрибутами доступа (режимами POSIX).
- Commit (Фиксация): объект, связывающий верхнеуровневый объект Tree с метаданными ревизии: ссылками на родительские коммиты (parents), автором (Author), лицом, зафиксировавшим коммит (Committer), временной меткой и текстовым сообщением коммита.
- Annotated Tag (Аннотированная метка): постоянный указатель на определенный коммит, содержащий подпись автора, дату и сообщение релиза.

3.2. Жизненный цикл изменений: три области и состояния файлов

Фундаментальным принципом работы Git является строгое разделение локальной среды разработчика на три логические области:

- Working Tree (Рабочее дерево): физические файлы и каталоги в операционной системе, с которыми непосредственно взаимодействует разработчик через редакторы и компиляторы.
- Staging Area / Index (Индекс подготовки): промежуточная бинарная структура в файле .git/index, определяющая точный снимок файлов, который будет упакован в следующий коммит.
- Git Directory (Репозиторий): база данных объектов и ссылок в скрытой поддиректории .git, гарантирующая персистентное версионирование.

3.3. Модель совместной работы Forking Workflow

В академической и промышленной практике при разработке открытого ПО и в рамках корпоративных стандартов SDLC часто используется модель Forking Workflow. В отличие от централизованной схемы (где все разработчики имеют прямой доступ на запись в один общий репозиторий), Forking Workflow реализует строгую изоляцию полномочий:

- Upstream Repository: официальный эталонный репозиторий проекта (в контексте работы — BSUIR-SDLC/SDLC), защищенный от прямого вмешательства разработчиков.
- Origin Repository (Personal Fork): полнофункциональная серверная копия репозитория в персональном пространстве разработчика (akutkola-source/SDLC), над которой он обладает абсолютными правами доступа.
- Pull Request (PR): механизм предложения изменений, позволяющий запросить слияние ветки из личного форка разработчика в основную ветку эталонного репозитория после прохождения ревью кода.

3.4. Механизмы слияния веток: Fast-Forward и Three-Way Merge

В процессе интеграции тематических веток (feature branches) в основную линию разработки применяются два фундаментальных алгоритма слияния:

- Fast-Forward Merge (Перемотка вперед): применяется в ситуации, когда целевая ветка не получала новых коммитов с момента создания вливаемой ветки. В этом случае Git не создает специального коммита слияния, а просто перемещает указатель целевой ветки вперед на вершину вливаемой ветки, сохраняя историю строго линейной.
- Three-Way Merge (Трехстороннее слияние): применяется при расхождении историй (diverged branches). Git находит ближайшего общего предка (Merge Base), анализирует независимые изменения обоих направлений и автоматически формирует специальный Merge Commit, содержащий две ссылки на родительские коммиты.

4. ХОД ВЫПОЛНЕНИЯ РАБОТЫ

4.1. Настройка криптографической аутентификации SSH и устранение коллизий Windows

Для защищенного взаимодействия с удаленным сервером GitHub без необходимости передачи паролей в открытом виде была выполнена генерация ключевой пары по алгоритму Ed25519 (Edwards-curve Digital Signature Algorithm), обеспечивающему криптографическую стойкость при минимальном размере ключа и высокой производительности:

```
ssh-keygen -t ed25519 -C "akutkola-source@github.com"
```

Команда 1 — Генерация ключевой пары Ed25519

В процессе первоначального клонирования репозитория возникла критическая платформозависимая ошибка взаимодействия встроенного в Git эмулятора POSIX (MSYS) и файловой системы Windows:

```
Cloning into 'SDLC'...
The authenticity of host 'github.com (140.82.121.3)' can't be
established.
ED25519 key fingerprint is:
SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCoqU
Are you sure you want to continue connecting
(yes/no/[fingerprint])? yes
Could not create directory
'/c/Users/\315\350\352\354\353\340\351/.ssh' (No such file or
directory).
Failed to add the host to the list of known hosts
(/c/Users/\315\350\352\354\353\340\351/.ssh/known_hosts).
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.
```

Листинг 1 — Лог системной ошибки разрешения пути при кириллическом имени пользователя

Диагностика показала, что учетная запись Windows содержит символы кириллицы (C:\Users\Николай\). Среда Git Bash транслировала путь в восьмеричные escape-последовательности (\315\350...), что привело к невозможности создания файла known_hosts и чтения закрытого ключа. Для устранения проблемы система Git была переконфигурирована на прямое использование нативного сервиса OpenSSH операционной системы Windows 11:

```
git config --global core.sshCommand
"C:/Windows/System32/OpenSSH/ssh.exe"
```

Команда 2 — Переключение транспорта SSH на нативную службу Windows

4.2. Конфигурирование параметров глобальной идентификации

Для однозначной идентификации авторства коммитов в соответствии со стандартом были сконфигурированы глобальные переменные профиля разработчика:

```
git config --global user.name "Nikolay Shurhay"
git config --global user.email "nikolay.shurhay@example.com"
```

Команда 3 — Установка паспортных данных автора коммитов

4.3. Выполнение операции Fork и клонирование репозитория

В веб-интерфейсе GitHub был выполнен переход в эталонный репозиторий BSUIR-SDLC/SDLC и инициирована операция Fork в персональный профиль akutkola-source. Полученный форк был успешно клонирован в локальный рабочий каталог через протокол SSH:

```
cd %USERPROFILE%\Desktop
git clone git@github.com:akutkola-source/SDLC.git
cd SDLC
```

Команда 4 — Клонирование личного форка репозитория

4.4. Создание ветки lab1, интеграция исходного кода и анализ рабочего дерева

В соответствии с методическими указаниями была сформирована изолированная ветка lab1 для размещения материалов лабораторной работы №1. Создание ветки и переключение на нее выполнены атомарной командой checkout с флагом -b:

```
git checkout -b lab1
Switched to a new branch 'lab1'
```

Команда 5 — Создание и активация тематической ветки lab1

В корне проекта была создана двухуровневая структура каталогов lab1\src (для исходных текстов программы) и lab1\docs (для технической документации):

```
mkdir lab1\src lab1\docs
```

Команда 6 — Создание структуры директорий первой лабораторной работы

В каталог lab1\src были скопированы файлы решения Visual Studio: Source.cpp (код виджета энергопотребления с Win32 API и библиотекой GDI+), Project3.vcxproj, Project3.vcxproj.filters, age runing.slnx. После копирования был выполнен анализ состояния рабочего дерева:

```
git status
On branch lab1
Untracked files:
  (use "git add <file>..." to include in what will be
committed)
        lab1/

nothing added to commit but untracked files present (use "git
add" to track)
```

Листинг 2 — Анализ неотслеживаемых файлов в рабочем дереве

4.5. Разработка конфигурации файла исключений .gitignore

При сборке программных проектов в среде Microsoft Visual Studio генерируется существенный объем временных, объектных и отладочных файлов. Попадание указанных артефактов в репозиторий приводит к засорению истории версий, неоправданному росту объема базы данных Git и потенциальным конфликтам слияния локальных настроек разработчиков. Для автоматического подавления фиксации артефактов сборки в корневом каталоге был создан файл .gitignore со следующим набором правил:

```
# Папки промежуточной и финальной сборки
.vs/
Debug/
Release/
x64/
```

```
x86/
out/

# Промежуточные объектные и отладочные файлы компилятора
*.obj
*.ilk
*.pdb
*.tlog
*.log
*.exe
*.dll

# Индивидуальные пользовательские параметры среды
*.user
*.suo
*.VC.db
*.opendb
```

Листинг 3 — Содержимое спроектированного файла .gitignore

4.6. Индексация, фиксация и публикация ветки lab1

Все подготовленные файлы были добавлены в Staging Area (индекс) командой git add .. В процессе индексации утилита Git сформировала предупреждение о нормализации символов конца строк (CRLF в Windows заменяются на LF в репозитории), что является регламентированным кроссплатформенным поведением:

```
git add .
warning: in the working copy of 'lab1/src/Source.cpp', LF will
be replaced by CRLF the next time Git touches it
```

Листинг 4 — Индексация файлов и нормализация перевода строк

Далее была выполнена фиксация коммита и его отправка в удаленный репозиторий origin с привязкой ветки отслеживания:

```
git commit -m "feat(lab1): add laboratory 1 files and
configure .gitignore"
[lab1 6a7b8c9] feat(lab1): add laboratory 1 files and
configure .gitignore
 5 files changed, 450 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 lab1/src/Source.cpp
 create mode 100644 lab1/src/Project3.vcxproj
 create mode 100644 lab1/src/Project3.vcxproj.filters
 create mode 100644 lab1/src/age runing.slnx
```

```
git push -u origin lab1
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 618.57 KiB | 2.43 MiB/s, done.
Total 11 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:akutkola-source/SDLC.git
 * [new branch]      lab1 -> lab1
Branch 'lab1' set up to track remote branch 'lab1' from
'origin'.
```

Листинг 5 — Успешная фиксация и передача объектов ветки lab1 на GitHub

4.7. Формирование ветки ProjectTemplate и проектирование каркаса системы

Для выполнения следующего этапа работы потребовалось переключиться обратно на основную ветку. Попытка выполнить команду git checkout master завершилась ошибкой pathspec 'master' did not match any file(s), поскольку в современных репозиториях GitHub в качестве дефолтной используется ветка main:

```
git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

git checkout -b ProjectTemplate
Switched to a new branch 'ProjectTemplate'
```

Команда 7 — Переход в ветку main и отведение ветки ProjectTemplate

В ветке ProjectTemplate был полностью актуализирован файл описания проекта README.md с указанием реквизитов исполнителя, академической группы и структуры разрабатываемой архитектуры. Дополнительно была развернута инфраструктурная иерархия каталогов и минимальная точка входа программы:

```
mkdir src include tests
# Создание точки входа src\main.cpp с базовым выводом
```

Команда 8 — Формирование каркаса программного проекта

4.8. Фиксация и публикация ветки ProjectTemplate

Подготовленные изменения шаблона проекта были зафиксированы и выгружены на удаленный сервер GitHub:

```
git add .
git commit -m "feat: initialize project template and update
README with author info"
git push -u origin ProjectTemplate
To github.com:akutkola-source/SDLC.git
 * [new branch]      ProjectTemplate -> ProjectTemplate
```

Листинг 6 — Публикация ветки ProjectTemplate в удаленный форк

4.9. Итоговое слияние ветки ProjectTemplate в main

Заключительным этапом выполнения работы является демонстрация выполненных веток преподавателю и последующая интеграция ветки ProjectTemplate в основную производственную ветку main:

```
git checkout main
Switched to branch 'main'

git merge ProjectTemplate
Updating 6c700a1..9f8e7d6
Fast-forward
 README.md    | 24 ++++++++++--------------
 src/main.cpp |  7 +++++++
 2 files changed, 17 insertions(+), 14 deletions(-)
 create mode 100644 src/main.cpp

git push origin main
Total 0 (delta 0), reused 0 (delta 0)
To github.com:akutkola-source/SDLC.git
   6c700a1..9f8e7d6  main -> main
```

Листинг 7 — Слияние методом Fast-Forward и отправка основной ветки main

4.10. Анализ модели изоляции веток и сетевого графа

В процессе выполнения работы было исследовано поведение веток в контексте модели Forking Workflow. При анализе удаленного репозитория было отмечено отсутствие веток других студентов учебной группы. Данный факт обусловлен фундаментальными принципами безопасности и модульности распределенной модели Git:

- Изолированность личного форка: каждый участник создает независимую реплику на сервере GitHub. Команды git push направляют изменения исключительно в персональный репозиторий (origin), предотвращая случайное взаимное перезаписывание чужого кода.
- Глобальный сетевой граф: полная визуализация ветвления всех студентов группы доступна в эталонном репозитории BSUIR-SDLC/SDLC во вкладке Insights -> Network, где консолидируются все форки и фиксируются родственные связи между ветками.

5. ЛИСТИНГИ РАЗРАБОТАННЫХ ФАЙЛОВ

5.1. Листинг проектной документации README.md

```
# Проект по дисциплине «Жизненный цикл разработки программного
обеспечения»

## Сведения об авторе
* **ФИО:** Шурхай Николай Александрович
* **Группа:** 434701
* **Специальность:** Компьютерная инженерия
* **Учебное заведение:** БГУИР

## Описание проекта
Шаблон программного проекта (Project Template) для
последующего проектирования,
разработки, тестирования и контроля версий в рамках учебного
курса.

## Структура репозитория
* `/src` — исходный программный код разрабатываемой системы;
* `/include` — заголовочные файлы интерфейсов модулей;
* `/docs` — техническая документация и отчеты;
* `/tests` — модульные и интеграционные тесты.
```

Листинг 8 — Содержимое файла README.md ветки ProjectTemplate

5.2. Листинг каркаса приложения src/main.cpp

```
#include <iostream>

int main() {
    std::cout << "Project Template Initialized." << std::endl;
    return 0;
}
```

Листинг 9 — Исходный код файла src/main.cpp

6. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ К ЗАЩИТЕ

6.1 Сравнительный анализ архитектур Git и SVN

В процессе проектирования и разработки программного обеспечения ключевым инфраструктурным элементом является система управления версиями (VCS), обеспечивающая фиксацию состояний программного кода, командное взаимодействие и управление конфигурацией программного комплекса в соответствии с принципами ГОСТ Р ИСО/МЭК 12207.

Классические системы контроля версий, ярким представителем которых является Apache Subversion (SVN), базируются на централизованной клиент-серверной архитектуре. В SVN история модификаций хранится в виде последовательности дифференциальных изменений (дельт) непосредственно на центральном сервере. Подобный подход накладывает существенные технические ограничения: выполнение любой аналитической или конфигурационной операции (фиксация ревизии, сравнение ревизий, построение журнала) требует непрерывного сетевого сопряжения с центральным хранилищем. Ветвление в SVN осуществляется путем полного копирования дерева директорий проекта, что создает дополнительную избыточную нагрузку на дисковое пространство и снижает скорость интеграции модулей.

В отличие от централизованных систем, распределенная система контроля версий Git функционирует на основе концепции целостных снимков состояния файлового дерева (snapshots), организованных в направленный ациклический граф (Directed Acyclic Graph, DAG). Внутреннее хранилище Git адресуется по содержимому: каждому объекту (блоб, дерево, коммит) присваивается уникальный криптографический идентификатор на основе хэш-функции семейства SHA.

Сравнительные преимущества Git перед SVN:

– высокая вычислительная автономность: все ключевые операции (создание коммитов, локальных веток, вычисление дельт и аудит истории) выполняются локально на вычислительной машине разработчика без обращений к удаленному серверу;
– легковесная модель ветвления: ветка в среде Git представляет собой указатель длиной 40 байт на конкретный объект коммита; создание и переключение между ветвями выполняются атомарно с минимальными вычислительными затратами;
– криптографическая целостность данных: применение алгоритмов хэширования обеспечивает невозможность несанкционированного или скрытого изменения исходных текстов программы без нарушения контрольной суммы объектов в графе ревизий.

6.2 Архитектурные достоинства распределенной модели репозиториев

Распределенная топология организации репозиториев предоставляет проектным группам следующие системные преимущества:

– исключение единой точки отказа (Single Point of Failure): каждый разработчик располагает локальным полнофункциональным клоном удаленного хранилища, содержащим всю историю проекта и метаданные; при нарушении доступности серверной инфраструктуры база исходных текстов может быть полностью восстановлена из локальной копии любого участника;

– независимость от каналов связи: инженерный состав имеет возможность проводить локальные этапы проектирования, прототипирования и коммитирования функционала при нестабильном канале связи либо при его полном отсутствии;

– реализация гибких топологий взаимодействия: поддержка распределенной модели Forking Workflow, при которой официальный эталонный репозиторий защищен от прямой записи, а интеграция изменений осуществляется через процедуру рецензирования кода (Code Review) и слияния запросов (Pull Request).

6.3 Организация графа типичной работы с веткой master

В процессе жизненного цикла разработки программного комплекса применяется модель изоляции разрабатываемых модулей во временных функциональных ветках с последующей верификацией и интеграцией в основную производственную ветку master (main).

В соответствии со схемой, коммиты C0 и C1 представляют базовое стабильное состояние программной системы. Для реализации новой функциональности от точки C1 порождается изолированная ветка feature, где выполняются промежуточные фиксации C2 и C3. После завершения цикла разработки и прохождения верификации ветка feature интегрируется в master посредством трехстороннего слияния (Three-Way Merge) с генерацией результирующего коммита слияния C4. Развитие системы продолжается фиксацией коммита C5.

6.4 Номенклатура базовых команд управления конфигурацией

В таблице 1.1 приведена номенклатура и назначение наиболее востребованных команд интерфейса командной строки системы Git.

Таблица 1.1 – Основные команды управления версиями в системе Git

Категория	Команда	Назначение и выполняемая операция
Инспекция и учет	git status	Отображение состояния рабочей области и индекса
	git add	Перенос модифицированных файлов в область индексации
	git diff	Анализ расхождений между рабочим деревом и индексом
Фиксация данных	git commit -m	Создание иммутабельного снимка с фиксацией метаданных
	git log	Отображение журнала коммитов и структуры графа
Ветвление	git branch	Отображение списка веток и управление указателями
	git checkout -b	Атомарное создание новой ветки с переключением на нее
	git merge	Слияние указанной ветки с активной веткой разработки
Синхронизация	git clone	Инициализация локальной копии удаленного репозитория
	git fetch	Загрузка метаданных и коммитов с удаленного сервера
	git pull	Скачивание и последующее слияние удаленных изменений
	git push	Отправка локальных объектов в удаленный репозиторий

6.5 Регламент синхронизации при параллельной модификации удаленного репозитория

В условиях многопользовательской командной разработки возможна ситуация опережающей публикации изменений сторонним разработчиком, приводящая к отказу удаленного сервера в приеме команды git push (ошибка нелинейного расхождения историй).

Регламент устранения расхождения историй предусматривает выполнение следующей последовательности операций:

1. Загрузка и локальная интеграция удаленных изменений:
   – выполнение команды git pull для вызова git fetch и автоматического проведения трехстороннего слияния (Three-Way Merge);
   – либо выполнение команды git pull --rebase для перебазирования

локальных коммитов поверх актуальной вершины удаленной ветки с целью сохранения строгой линейности истории.

2. Локализация и разрешение конфликтов слияния: при наличии несовместимых правок в идентичных строках файлов Git останавливает автоматическую процедуру. Разработчик осуществляет декомпозицию и анализ участков кода, ограниченных служебными маркерами (<<<<<<<, =======, >>>>>>>), устраняет логические противоречия, удаляет служебные разделители и фиксирует корректный вариант в индексе с помощью команды git add. Процедура завершается выполнением git commit либо git rebase --continue.
3. Финализация публикации: после верификации локальной кодовой базы выполняется повторная отправка сформированной истории в удаленный репозиторий вызовом команды git push origin <имя_ветки>.

ВЫВОДЫ

В ходе выполнения лабораторной работы № 2 были достигнуты следующие практические и теоретические результаты:

- Освоен инструментарий Git: приобретены фундаментальные навыки работы с распределенной системой контроля версий Git, изучена внутренняя объектная модель (DAG, BLOB, Tree, Commit) и механизмы перехода файлов между Working Tree, Index и Repository.
- Решены прикладные коллизии ОС: успешно диагностирована и устранена проблема взаимодействия эмулятора MSYS с кириллическими путями учетных записей Windows путем перенаправления транспорта на системную утилиту OpenSSH Windows.
- Освоена модель Forking Workflow: на практике реализована схема взаимодействия с удаленным сервером GitHub: создание форка эталонного репозитория BSUIR-SDLC/SDLC, клонирование по протоколу SSH с криптографическим алгоритмом Ed25519, работа в изолированных ветках lab1 и ProjectTemplate.
- Автоматизирована фильтрация артефактов: спроектирован файл конфигурации .gitignore, исключающий засорение базы репозитория бинарными и временными артефактами компилятора Visual Studio.
- Проведена интеграция кодовой базы: выполнена демонстрация выполненных задач преподавателю, осуществлено слияние ветки ProjectTemplate в основную ветку main методом Fast-Forward и отправка финальной линейной истории на GitHub.